Graphs, Trees, Sorting, and Time Complexity

1. Graphs

A graph is a collection of nodes (vertices) and edges (links) that connect pairs of nodes. Graphs can be used to model many real-world scenarios, such as social networks, maps, and network topologies.

Types of Graphs:

- Undirected Graph: Edges have no direction.
- Directed Graph (Digraph): Edges have direction.
- Weighted Graph: Edges have weights, representing the cost to traverse them.
- Connected Graph: There's a path between every pair of vertices.
- Acyclic Graph: A graph with no cycles. If it's directed and acyclic, it's called a Directed Acyclic Graph (DAG).

Common Graph Algorithms:

- Breadth-First Search (BFS): Explores nodes level by level, used to find the shortest path in unweighted graphs.
- Depth-First Search (DFS): Explores as far as possible down a branch before backtracking, useful for topological sorting and detecting cycles.
- Dijkstra's Algorithm: Finds the shortest path in a weighted graph with non-negative weights.
- Kruskal's and Prim's Algorithms: Used to find the Minimum Spanning Tree (MST) in a graph.

2. Trees

A tree is a special type of graph with no cycles, where any two nodes are connected by exactly one path. Trees are hierarchical structures and are widely used in various applications.

Types of Trees:

- Binary Tree: Each node has at most two children (left and right).
- Binary Search Tree (BST): A binary tree where the left subtree contains only nodes with values less than the parent node, and the right subtree only nodes with values greater.
- AVL Tree: A self-balancing BST where the difference between heights of left and right subtrees is at most one.
- Red-Black Tree: A self-balancing BST with an additional property that the tree is balanced with the help of "coloring" nodes.

- Trie: A tree used for efficient retrieval of keys in datasets like dictionaries.
- Heap: A special tree-based data structure that satisfies the heap property (max-heap or min-heap).

Tree Traversal Methods:

- In-order Traversal: Left, Root, Right (used in BST to retrieve sorted data).
- Pre-order Traversal: Root, Left, Right (used to create a copy of the tree).
- Post-order Traversal: Left, Right, Root (used to delete the tree).
- Level-order Traversal: Visit nodes level by level.

3. Sorting Algorithms

Sorting algorithms arrange elements in a list in a certain order (ascending or descending).

Common Sorting Algorithms:

- Bubble Sort: Repeatedly steps through the list, compares adjacent elements, and swaps them if they are in the wrong order.
- Selection Sort: Divides the list into sorted and unsorted parts, repeatedly selecting the smallest (or largest) element from the unsorted part and moving it to the sorted part.
- Insertion Sort: Builds the sorted list one item at a time, inserting each new item into its correct position.
- Merge Sort: Divides the list into halves, sorts each half, and then merges the sorted halves.
- Quick Sort: Selects a 'pivot' element, partitions the list around the pivot, and recursively sorts the partitions.
- Heap Sort: Converts the list into a heap and repeatedly extracts the maximum (or minimum) element to build the sorted list.
- Radix Sort: Sorts elements by processing individual digits.

Stability and In-Place:

- Stable Sorting: Preserves the relative order of equal elements.
- In-Place Sorting: Uses a constant amount of extra space.

4. Time Complexity

Time complexity measures the amount of time an algorithm takes to run as a function of the length of the input. It's commonly expressed using Big-O notation.

Common Big-O Notations:

- O(1): Constant time - the algorithm's runtime does not depend on the input size.
- O(log n): Logarithmic time - typically seen in binary search algorithms.
- O(n): Linear time - the runtime grows linearly with the input size.
- O(n log n): Linearithmic time - often seen in efficient sorting algorithms like merge sort and quicksort.
- O(n^2): Quadratic time - often seen in simple sorting algorithms like bubble sort, selection

sort, and insertion sort.
- O(2^n): Exponential time - seen in algorithms with a brute-force approach to solving problems.
- O(n!): Factorial time - seen in algorithms that generate all permutations of an input set.

Space Complexity:

It measures the amount of memory space an algorithm uses relative to the input size.